

iOS build přes GitHub Actions z Windows

Univerzální návod pro Flutter aplikace · Generováno na základě setupu projektu TORKIS · 2026-05-15

Obsah

- 1. Co budeš potřebovat (prerekvizity)
- 2. Apple Developer — vytvoření certifikátu, App ID a provisioning profilu
- 3. App Store Connect — registrace aplikace a API key
- 4. Příprava souborů na Windows (.p12 z certifikátu)
- 5. GitHub Secrets — seznam 9 hodnot
- 6. Workflow soubor (kompletní YAML)
- 7. PowerShell pomocný skript
- 8. Úprava iOS projektu (project.pbxproj, Info.plist, ikona)
- 9. První spuštění a TestFlight
- 10. Troubleshooting nejčastějších chyb
- 11. Checklist pro nový projekt

1) Co budeš potřebovat

Položka	Kde získat
Apple Developer Program (\$99/rok)	developer.apple.com → Enroll
App Store Connect přístup	součást Developer programu
GitHub repozitář s Flutter projektem	libovolný účet, free pro veřejné repos
Bundle ID (např. <code>com.firma.app</code>)	navrhneš si sám, musí být unikátní
Windows PC s nainstalovaným:	
• Flutter SDK	flutter.dev/docs/get-started/install
• Git for Windows (obsahuje openssl)	git-scm.com
• PowerShell 5.1+ (součást Windows)	—

Mac nepotřebuješ. Veškerý iOS build běží na macOS runnerech v GitHub Actions. Z Windows jenom připravíš certifikáty, profily a zatlačíš to do Gitu.

2) Apple Developer — certifikát, App ID a provisioning profile

2.1 App ID (Identifier)

1. Jdi na **developer.apple.com/account/resources/identifiers**
2. Klikni + → App IDs → App → Continue
3. Description: libovolný popis (např. "Moje aplikace")
4. Bundle ID: **Explicit** → zadej např. `com.firma.app` (musí odpovídat tomu v `ios/Runner.xcodeproj/project.pbxproj` a `pubspec.yaml`)
5. Capabilities: zaškrtni jen to, co aplikace skutečně používá (Push Notifications, Sign in with Apple, atd.)
6. Continue → Register

2.2 Distribution certifikát (.cer + .p12)

Apple vyžaduje, abys měl **Apple Distribution** certifikát pro podpis IPA. Z Macu se vyrobí standardně přes Keychain Access. **Z Windows** potřebuješ obejít CSR (Certificate Signing Request) přes `openssl` :

```
# Vytvoř složku pro signing materiály mimo repo
mkdir $env:USERPROFILE\.ios-signing
cd $env:USERPROFILE\.ios-signing

# Cesta k openssl z Git for Windows
$openssl = "C:\Program Files\Git\usr\bin\openssl.exe"

# 1) Vygeneruj privátní klíč
& $openssl genrsa -out distribution.key 2048

# 2) Vytvoř CSR (Certificate Signing Request)
& $openssl req -new -key distribution.key -out distribution.csr `
  -subj "/emailAddress=tvuj@email.cz/CN=Tvoje Jmeno/C=CZ"
```

Pak v Apple Developer portálu:

1. **developer.apple.com/account/resources/certificates** → +
2. Vyber **Apple Distribution** → Continue
3. Upload `distribution.csr` → Continue
4. Download `distribution.cer` → ulož do `~/.ios-signing/`

Teď spoj cert s klíčem do `.p12` :

```
& $openssl pkcs12 -export `
  -out distribution.p12 `
  -inkey distribution.key `
  -in distribution.cer
# openssl se zeptá na heslo – zvol něco silného a poznamenej si
```

Heslo k .p12 si poznamenej! Pokud ho zapomeneš, můžeš `.p12` kdykoliv znovu vygenerovat z `.key` + `.cer` (které zůstávají v složce).

2.3 Provisioning Profile (App Store distribution)

1. **developer.apple.com/account/resources/profiles** → +

2. Sekce **Distribution** → **App Store** → Continue
3. App ID: vyber to, co jsi vytvořil v kroku 2.1
4. Certificate: zaškrtni svůj **Apple Distribution** cert
5. Provisioning Profile Name: např. `MyApp_AppStore`
6. Generate → Download `.mobileprovision` do `Downloads`

Pokud máš víc Distribution certifikátů (Apple povoluje max 3), **zaškrtni všechny** — vyhneš se chybě "Profile doesn't include signing certificate" pokud v budoucnu vytvoříš nový.

3) App Store Connect — aplikace a API key

3.1 Vytvoř aplikaci v ASC

Důležité: Aplikaci v App Store Connect *musíš vytvořit dřív, než pošleš první build*, jinak Apple build "ztratí" a v TestFlight nic neuvidíš.

1. appstoreconnect.apple.com → **My Apps** → + → **New App**
2. Platforms: iOS
3. Bundle ID: vyber ten z kroku 2.1
4. Name: zobrazované jméno (max 30 znaků)
5. Primary Language, SKU (libovolný unikátní řetězec)
6. Create

3.2 App Store Connect API Key (.p8)

1. appstoreconnect.apple.com/access/integrations/api → tab **Keys** → +
2. Name: např. "GitHub Actions"
3. Access: **App Manager**
4. Generate → **Download API Key** (jen jednorázové stažení, ulož bezpečně)
5. Poznamenej si **Key ID** (10 znaků) a **Issuer ID** (UUID v záhlaví)

4) Příprava souborů na Windows

Měl bys teď mít:

- `~/ios-signing/distribution.p12` (+ heslo)
- `~/ios-signing/distribution.key`, `distribution.cer` (zachovat pro budoucí re-generaci)
- `~/Downloads/MyApp_AppStore.mobileprovision`
- `~/Downloads/AuthKey_XXXXXXXXXX.p8`
- Key ID (10 znaků), Issuer ID (UUID), Apple Team ID (10 znaků — najdeš v developer.apple.com → Membership)

5) GitHub Secrets — 9 hodnot

V repu jdi do **Settings** → **Secrets and variables** → **Actions** → **New repository secret**:

Secret	Hodnota
<code>IOS_CERT_BASE64</code>	obsah <code>distribution.p12</code> v base64
<code>IOS_CERT_PASSWORD</code>	heslo k <code>.p12</code>
<code>KEYCHAIN_PASSWORD</code>	libovolné heslo (např. <code>gh-actions-keychain</code>)
<code>IOS_PROFILE_BASE64</code>	obsah <code>.mobileprovision</code> v base64
<code>APPLE_TEAM_ID</code>	10znakové Team ID (např. <code>CZRDLTZC6L</code>)
<code>ASC_KEY_ID</code>	Key ID z App Store Connect (10 znaků)
<code>ASC_ISSUER_ID</code>	Issuer ID (UUID)
<code>ASC_KEY_BASE64</code>	obsah <code>AuthKey_XXX.p8</code> v base64

Sdíleno napříč všemi tvými aplikacemi (stačí jeden Apple Developer účet): `IOS_CERT_*` , `KEYCHAIN_PASSWORD` , `APPLE_TEAM_ID` , `ASC_*` . Specifické pro každou app: jen `IOS_PROFILE_BASE64` .

6) Workflow soubor

Vytvoř soubor `.github/workflows/ios-testflight.yml` :

```

name: iOS Build

on:
  workflow_dispatch:

jobs:
  build:
    runs-on: macos-15

    steps:
      - uses: actions/checkout@v4

      - uses: maxim-lobanov/setup-xcode@v1
        with:
          xcode-version: latest-stable

      - uses: subosito/flutter-action@v2
        with:
          channel: stable
          cache: true

      - name: Flutter pub get
        run: flutter pub get

      - name: Import signing certificate
        env:
          CERT_BASE64: ${ secrets.IOS_CERT_BASE64 }
          CERT_PASSWORD: ${ secrets.IOS_CERT_PASSWORD }
          KEYCHAIN_PASSWORD: ${ secrets.KEYCHAIN_PASSWORD }
        run: |
          CERT=$RUNNER_TEMP/cert.p12
          KCH=$RUNNER_TEMP/app.keychain-db
          echo -n "$CERT_BASE64" | base64 --decode -o $CERT
          security create-keychain -p "$KEYCHAIN_PASSWORD" $KCH
          security set-keychain-settings -lut 21600 $KCH
          security unlock-keychain -p "$KEYCHAIN_PASSWORD" $KCH
          security import $CERT -P "$CERT_PASSWORD" -A -t cert -f pkcs12 -k $KCH
          security set-key-partition-list -S apple-tool:,apple: \
            -k "$KEYCHAIN_PASSWORD" $KCH
          security list-keychain -d user -s $KCH

      - name: Install provisioning profile
        env:
          PROFILE_BASE64: ${ secrets.IOS_PROFILE_BASE64 }
        run: |
          PP=$RUNNER_TEMP/profile.mobileprovision
          printf '%s' "$PROFILE_BASE64" | tr -d ' \n\r\t' | base64 --decode > $PP
          PP_UUID=$(security cms -D -i $PP | plutil -extract UUID raw -)
          echo "PP_UUID=$PP_UUID" >> $GITHUB_ENV
          mkdir -p ~/Library/MobileDevice/Provisioning\ Profiles
          cp $PP ~/Library/MobileDevice/Provisioning\ Profiles/$PP_UUID.mobileprovision

      - name: Inject signing into xcconfig
        env:

```

```

TEAM_ID: ${ secrets.APPLE_TEAM_ID }
run: |
{
    echo "CODE_SIGN_STYLE=Manual"
    echo "DEVELOPMENT_TEAM=$TEAM_ID"
    echo "CODE_SIGN_IDENTITY=Apple Distribution"
    echo "PROVISIONING_PROFILE_SPECIFIER=$PP_UUID"
} >> ios/Flutter/Release.xcconfig

- name: Create ExportOptions.plist
env:
    TEAM_ID: ${ secrets.APPLE_TEAM_ID }
    BUNDLE_ID: com.firma.app # <-- ZMĚŇ na svůj bundle ID
run: |
    cat > ios/ExportOptions.plist << PLIST
    <?xml version="1.0" encoding="UTF-8"?>
    <!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
    "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
    <plist version="1.0"><dict>
    <key>method</key><string>app-store-connect</string>
    <key>teamID</key><string>${TEAM_ID}</string>
    <key>signingStyle</key><string>manual</string>
    <key>signingCertificate</key><string>Apple Distribution</string>
    <key>provisioningProfiles</key><dict>
    <key>${BUNDLE_ID}</key><string>${PP_UUID}</string>
    </dict>
    <key>uploadBitcode</key><false/>
    <key>uploadSymbols</key><true/>
    </dict></plist>
    PLIST

- name: Build IPA
run: |
    BUILD_NUMBER=$((100 + ${ github.run_number } ))
    flutter build ipa --release \
        --export-options-plist=ios/ExportOptions.plist \
        --build-number=$BUILD_NUMBER

- name: Upload to TestFlight
env:
    ASC_KEY_ID: ${ secrets.ASC_KEY_ID }
    ASC_ISSUER_ID: ${ secrets.ASC_ISSUER_ID }
    ASC_KEY_BASE64: ${ secrets.ASC_KEY_BASE64 }
run: |
    KEY=$RUNNER_TEMP/AuthKey_${ASC_KEY_ID}.p8
    echo -n "$ASC_KEY_BASE64" | base64 --decode -o $KEY
    mkdir -p ~/.appstoreconnect/private_keys
    cp $KEY ~/.appstoreconnect/private_keys/
    IPA=$(find build/ios/ipa -name '*.ipa' | head -1)
    set -o pipefail
    xcrun altool --upload-app -t ios -f "$IPA" \
        --apiKey "$ASC_KEY_ID" \
        --apiIssuer "$ASC_ISSUER_ID" 2>&1 | tee /tmp/altool.log
    if grep -qE "UPLOAD FAILED|ERROR:|Validation failed" /tmp/altool.log; then
        echo "::error::Apple altool reported failure"
    fi

```

```

        exit 1
    fi

    - uses: actions/upload-artifact@v4
      if: always()
      with:
        name: ipa-${{ github.run_number }}
        path: build/ios/ipa/*.ipa
        retention-days: 14

```

7) PowerShell pomocný skript

Vytvoř `scripts/prepare-ios-secrets.ps1` — vygeneruje base64 hodnoty do textových souborů, ze kterých zkopíruješ do GitHub Secrets:

```

param(
    [Parameter(Mandatory=$true)][string]$ProfilePath,
    [Parameter(Mandatory=$true)][string]$P8Path,
    [string]$P12Path = "$env:USERPROFILE\.ios-signing\distribution.p12"
)
$OutDir = "$env:USERPROFILE\.ios-signing\github-secrets"
New-Item -ItemType Directory -Force -Path $OutDir | Out-Null

function ToBase64($path) {
    [Convert]::ToBase64String([IO.File]::ReadAllBytes($path))
}

function Save-Secret($name, $value) {
    [IO.File]::WriteAllText((Join-Path $OutDir "$name.txt"), $value)
    Write-Host "  $name -> $OutDir\$name.txt" -ForegroundColor Green
}

Save-Secret "IOS_CERT_BASE64"      (ToBase64 $P12Path)
Save-Secret "IOS_PROFILE_BASE64"   (ToBase64 $ProfilePath)
Save-Secret "ASC_KEY_BASE64"       (ToBase64 $P8Path)
Save-Secret "ASC_KEY_ID" (
    [IO.Path]::GetFileNameWithoutExtension($P8Path) -replace '^AuthKey_', ''
)

$text = [Text.Encoding]::UTF8.GetString([IO.File]::ReadAllBytes($ProfilePath))
if ($text -match '<key>application-identifier</key>\s*<string>([A-Z0-9]+)\.') {
    Save-Secret "APPLE_TEAM_ID" $matches[1]
}

Start-Process explorer.exe $OutDir
Write-Host ""
Write-Host "Rucne dopln tyto 3 secrets:" -ForegroundColor Magenta
Write-Host "  IOS_CERT_PASSWORD -> heslo k .p12"
Write-Host "  KEYCHAIN_PASSWORD -> libovolne"
Write-Host "  ASC_ISSUER_ID      -> z App Store Connect (Issuer ID)"

```

Spuštění:

```
.\scripts\prepare-ios-secrets.ps1 `
-ProfilePath "$env:USERPROFILE\Downloads\MyApp_AppStore.mobileprovision" `
-P8Path "$env:USERPROFILE\Downloads\AuthKey_XXXXXXXXXX.p8"
```

8) Úpravy iOS projektu (jednorázové)

8.1 project.pbxproj — odstranit Automatic signing pro Runner

V `ios/Runner.xcodeproj/project.pbxproj` najdi všechny řádky obsahující `CODE_SIGN_STYLE = Automatic;` v sekcích Runner targetu (Debug, Release, Profile — typicky 3 výskyty u **Runner**, navíc 3 u **RunnerTests** — ty **nech**). Řádky u Runneru smaž.

Runner targety jsou ty, kde je `PRODUCT_BUNDLE_IDENTIFIER = com.firma.app`. RunnerTests targety mají `PRODUCT_BUNDLE_IDENTIFIER = com.firma.app.RunnerTests` — těch se nedotýkej.

8.2 Info.plist — App Encryption

Aby se Apple Connect po každém uploadu neptal na export compliance, přidej do `ios/Runner/Info.plist` (kdekoliv mezi `<dict>` a `</dict>`):

```
<key>ITSAppUsesNonExemptEncryption</key>
<false/>
```

Platí, pokud aplikace používá jen HTTPS přes iOS SDK, Apple Keychain, Face/Touch ID a žádné vlastní šifrování.

8.3 App ikona — bez alpha kanálu

Apple odmítá 1024×1024 ikony s transparency. PowerShell flatten s brand barvou:

```
Add-Type -AssemblyName System.Drawing
$src = "assets/images/app-icon-1024.png"
$img = [System.Drawing.Image]::FromFile($src)
$flat = New-Object System.Drawing.Bitmap 1024,1024,`
    ([System.Drawing.Imaging.PixelFormat]::Format24bppRgb)
$g = [System.Drawing.Graphics]::FromImage($flat)
$g.Clear([System.Drawing.ColorTranslator]::FromHtml('#0B1A2E'))
$g.DrawImage($img, 0, 0, 1024, 1024)
$flat.Save($src, [System.Drawing.Imaging.ImageFormat]::Png)
$img.Dispose(); $flat.Dispose()
```

Pak regeneruj iOS asset catalog:

```
dart run flutter_launcher_icons
```

9) První spuštění a TestFlight

1. Commit + push všeho do `main`
 2. GitHub → **Actions** → **iOS Build** → **Run workflow** → Run
 3. Build trvá 15–25 min (první pod install je nejpomalejší)
 4. Po úspěšném upload zkontroluj **App Store Connect** → **Activity** → **All Builds** — build je nejdřív "Processing" (5–30 min)
 5. Po zpracování: **TestFlight** → **Builds** → **klik na build** → **vyřeš Export Compliance** (pokud nemáš v Info.plist klíč z 8.2)
 6. Přidej se do **Internal Testing** skupiny → Apple ti pošle email s redeem code
 7. Na iPhoneu otevři **TestFlight app** → profil → **Redeem** → zadej kód
 8. Install → testuj
-

10) Troubleshooting

CHYBA: "RUNNER HAS CONFLICTING PROVISIONING SETTINGS. RUNNER IS AUTOMATICALLY SIGNED, BUT PROVISIONING PROFILE * HAS BEEN MANUALLY SPECIFIED."**

Příčina: `project.pbxproj` má v Runner targetu `CODE_SIGN_STYLE = Automatic`.

Řešení: Krok 8.1 — smaž tyto řádky.

CHYBA: "PROVISIONING PROFILE DOESN'T INCLUDE SIGNING CERTIFICATE APPLE DISTRIBUTION"

Příčina: Tvůj `.p12` obsahuje jiný Distribution cert než ten, který je v provisioning profilu.

Řešení: V Apple Developer portálu otevři profile → Edit → zaškrtni všechny tvoje Distribution certifikáty → Save → stáhni nový `.mobileprovision` → aktualizuj `IOS_PROFILE_BASE64` v GitHub Secrets.

CHYBA: "NO PROFILE FOR TEAM '**' MATCHING '****' FOUND"**

Příčina: Xcode hledá profile podle jména, ale jméno se mezerou (např. "MyApp App Store") může selhat.

Řešení: Workflow používá UUID místo jména (`$PP_UUID` v `xconfig` i `ExportOptions.plist`) — ujisti se, že máš to ve workflow.

CHYBA: "SECURITY: FAILED TO ADD DATA TO DECODER: UNKNOWN (-8183)"

Příčina: Secret `IOS_PROFILE_BASE64` má whitespace/newline na konci nebo není správně aktualizovaný.

Řešení: Ve workflow je `tr -d ' \n\r\t'` pipe co odstraní whitespace. Pokud chyba přetrvává, znovu vlož secret přes: `Get-Content secret.txt -Raw | Set-Clipboard` a Ctrl+V v GitHub UI.

CHYBA: "INVALID LARGE APP ICON. THE LARGE APP ICON CAN'T BE TRANSPARENT OR CONTAIN AN ALPHA CHANNEL."

Příčina: 1024×1024 ikona má alpha kanál.

Řešení: Krok 8.3 — PowerShell flatten + regenerace přes `flutter_launcher_icons`.

CHYBA: BUILD SE NAHRÁL, ALE V TESTFLIGHT "NO BUILDS"

Příčina A: Apple build odmítl po nahrání (validation error). Workflow ale ukázal success protože `xcrun altool` vrátil exit 0. Workflow ve verzi z kroku 6 už toto detekuje a failuje.

Příčina B: Aplikace v App Store Connect neexistovala v době uploadu — Apple build "ztratil". Vytvoř app v ASC a spusť workflow znovu (build number se sám zvýší).

Příčina C: "Missing Compliance" — vyřeš v ASC → TestFlight → klik na build → vyber "None of the algorithms" → Save.

CHYBA: "INVALID BINARY. THE BUNDLE CONTAINS A DUPLICATE BUILD NUMBER."

Příčina: Build s tímto číslem už byl jednou nahrán.

Řešení: Workflow ve verzi z kroku 6 používá `BUILD_NUMBER=$((100 + github.run_number))` — každé spuštění má unikátní číslo automaticky.

CHYBA: "NO APPLICATION RECORDS WERE FOUND WITH THE SPECIFIED BUNDLE IDENTIFIER"

Příčina: App v ASC neexistuje nebo byla vytvořena až po uploadu.

Řešení: ASC → My Apps → +. Pak workflow znovu.

11) Checklist pro nový projekt

- ☐ Bundle ID rozhodnutý a unikátní
- ☐ App ID vytvořeno v Apple Developer
- ☐ Distribution .p12 + heslo
- ☐ App Store provisioning profile
- ☐ App Store Connect API key (.p8) + Key ID + Issuer ID
- ☐ Apple Team ID poznamenán
- ☐ Aplikace vytvořená v App Store Connect *před* prvním uploadem
- ☐ 9 secrets v GitHub repu
- ☐ `.github/workflows/ios-testflight.yml` ve repu
- ☐ `project.pbxproj` bez `CODE_SIGN_STYLE = Automatic` u Runner targetu
- ☐ `Info.plist` s `ITSAAppUsesNonExemptEncryption = false`
- ☐ App ikona 1024×1024 bez alpha kanálu
- ☐ `pubspec.yaml` má rozumnou marketing version (např. `1.0.0`)
- ☐ Bundle ID v `ExportOptions.plist` dictionary klíči odpovídá tvému App ID
- ☐ TestFlight app na iPhoneu nainstalovaná, Apple ID matchuje email testera

Až bude všechno odškrtnuto: Actions → Run workflow → ~20 min → email od Apple → TestFlight redeem → testuj.